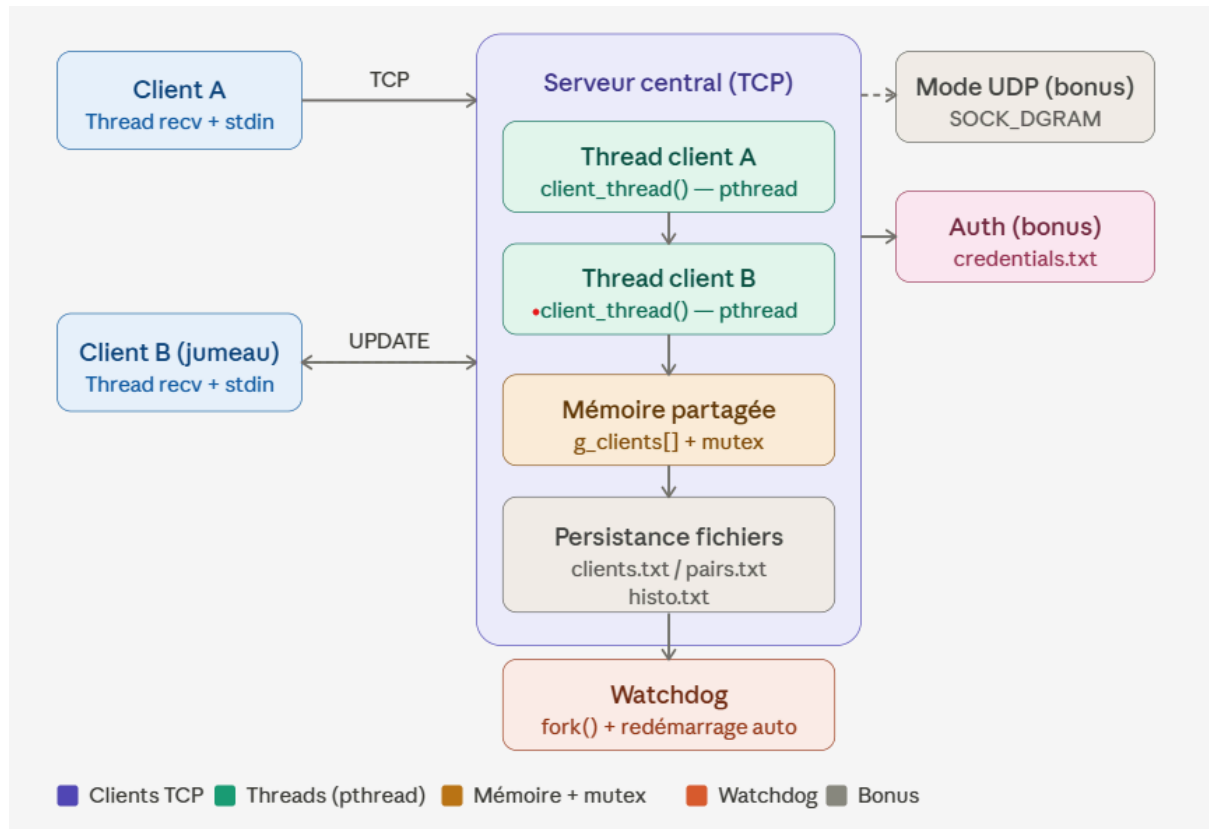


Explication de QUANTUM TWIN — server.c & client.c

Vue d'ensemble

Ce projet implémente un système de **jumeaux numériques** (*digital twins*) : deux clients peuvent être appariés, et quand l'un change d'état, l'autre est automatiquement notifié.



(Clique sur les blocs pour approfondir un sujet)

Explication détaillée

server.c

Le serveur est organisé en 10 sections bien séparées. Voici les parties importantes :

Section 1 — Journalisation : `log_info()` et `log_warn()` sont de simples wrappers autour de `printf` qui ajoutent un horodatage. Rien de complexe, mais utile pour le débogage.

Section 2 — Persistance : Trois fichiers sont utilisés pour sauvegarder l'état entre les redémarrages. `save_clients()` écrit la table en mémoire vers `clients.txt`,

`append_histo()` ajoute une ligne à `histo.txt`, et `find_twin()` / `add_pair()` lisent et écrivent `pairs.txt` pour gérer les appariements.

Section 3 — Table en mémoire : `g_clients[]` est un tableau global qui représente tous les clients connectés. Les fonctions `register_client()`, `update_state()`, `disconnect_client()` et `send_to_client()` le manipulent.

Section 4 — Authentification : `authenticate()` lit `credentials.txt` ligne par ligne et vérifie le couple username/password. Si le fichier n'existe pas, tout le monde est accepté sans mot de passe.

Section 5 — Protocole : C'est le cœur de la logique métier. Chaque commande (`CONNECT`, `STATE`, `SYNC`, `MSG`, `LIST`) a sa propre fonction `handle_*`. La commande `STATE` est la plus importante : elle met à jour l'état du client, cherche son jumeau via `find_twin()`, et lui envoie un message `UPDATE` s'il est connecté.

Section 6 — Thread client : La fonction `client_thread()` est la boucle principale de traitement d'un client. Elle lit les messages en boucle avec `recv()`, découpe la commande et les arguments, puis appelle le bon `handle_*`.

Section 7 — Boucle TCP : `run_tcp()` crée la socket d'écoute, attend les connexions avec `accept()`, et pour chaque nouvelle connexion alloue un `ThreadArg` (avec `calloc`) puis crée un thread détaché avec `pthread_create()`.

Section 8 — UDP (bonus) : `run_udp()` est une boucle simple sans threads, qui répond à `PING` et `STATE` via `recvfrom()` / `sendto()`.

Section 9 — Watchdog : `run_with_watchdog()` utilise `fork()`. Le processus parent attend que l'enfant (le vrai serveur) se termine, et s'il a crashé (signal reçu, code de retour non nul), il le relance après 2 secondes.

client.c

Beaucoup plus simple. Il a deux parties :

`run_tcp()` crée une socket, se connecte, puis lance un **thread de réception** (`recv_thread`) qui affiche en arrière-plan tout ce que le serveur envoie. Le thread principal lui, lit l'entrée clavier (`fgets`) et envoie chaque ligne au serveur.

`run_udp()` est encore plus simple : envoi/réception synchrones, avec un timeout de 2 secondes sur le `recv`.

Où sont utilisés les multithreads, sockets et la synchronisation ?

Multithreads (pthread)

Endroit	Rôle
<code>run_tcp()</code> dans <code>server.c</code>	<code>pthread_create()</code> + <code>pthread_detach()</code> — un thread par client accepté
<code>client_thread()</code> dans <code>server.c</code>	La fonction exécutée par chaque thread, boucle <code>recv()</code> infinie
<code>run_tcp()</code> dans <code>client.c</code>	<code>pthread_create()</code> — thread de réception en arrière-plan
<code>recv_thread()</code> dans <code>client.c</code>	Affiche les messages serveur pendant que le main lit le clavier

Le serveur est donc **massivement parallèle** : 64 clients peuvent être traités simultanément, chacun dans son propre thread.

Sockets

Endroit	Détail
<code>run_tcp()</code> serveur	<code>socket()</code> + <code>bind()</code> + <code>listen()</code> + <code>accept()</code> — socket TCP d'écoute
<code>run_udp()</code> serveur	<code>socket(SOCK_DGRAM)</code> + <code>bind()</code> + <code>recvfrom()</code> / <code>sendto()</code>
<code>run_tcp()</code> client	<code>socket()</code> + <code>connect()</code> + <code>send()</code> / <code>recv()</code>
<code>run_udp()</code> client	<code>socket(SOCK_DGRAM)</code> + <code>sendto()</code> / <code>recvfrom()</code>
<code>send_to_client()</code> serveur	Envoie vers la socket d'un client à partir de son <code>sockfd</code> stocké en mémoire

Synchronisation (mutex)

C'est la partie la plus délicate car plusieurs threads accèdent aux mêmes ressources en même temps.

Mutex	Protège quoi	Utilisé où
<code>mtx_mem</code>	Le tableau <code>g_clients[]</code> et <code>g_nb_clients</code>	<code>get_client()</code> , <code>register_client()</code> , <code>update_state()</code> , <code>disconnect_client()</code> , <code>send_to_client()</code> , <code>handle_list()</code>
<code>mtx_clients</code>	Les lectures/écritures sur <code>clients.txt</code>	<code>save_clients()</code> , <code>load_clients()</code>

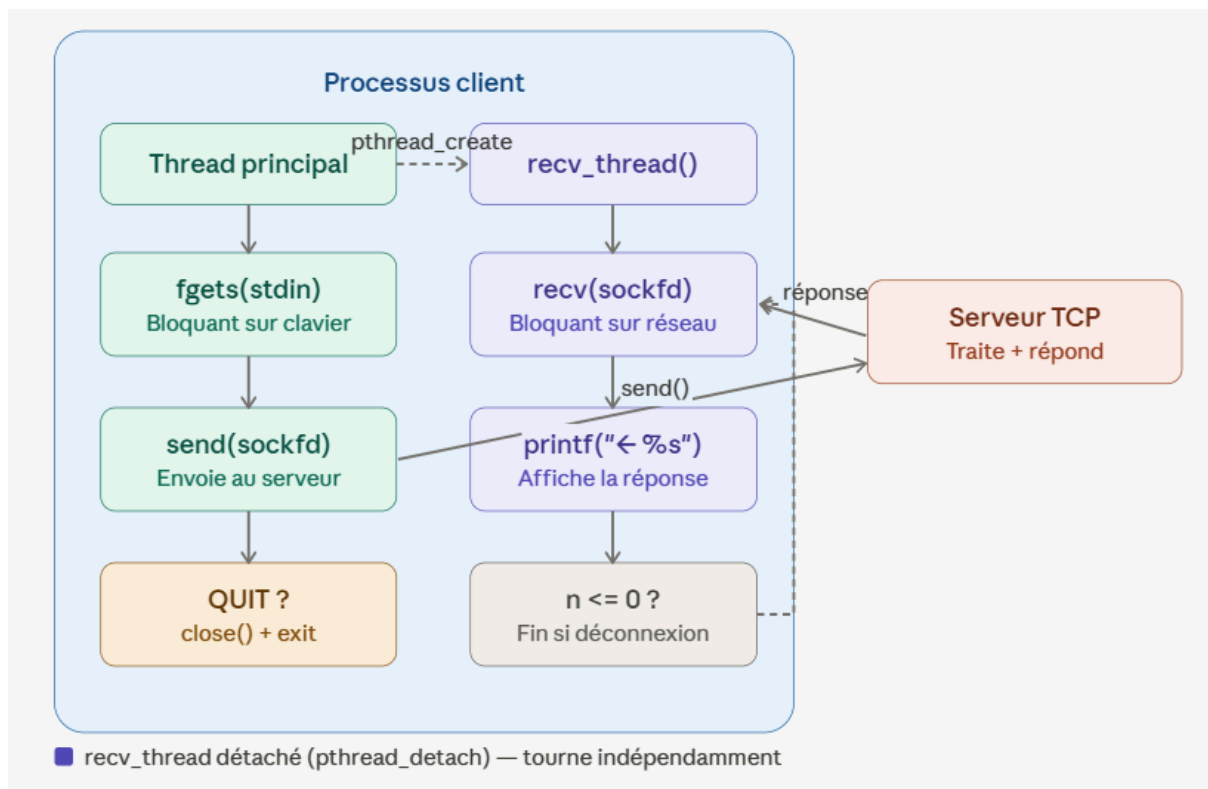
`mtx_pair` Les lectures/écritures `find_twin(), add_pair()`
sur `pairs.txt`

`mtx_histo` Les écritures sur `append_histo()`
sur `histo.txt`

Le pattern est toujours le même : `pthread_mutex_lock()` juste avant la section critique, `pthread_mutex_unlock()` juste après. Sans ces mutex, deux threads pourraient modifier `g_clients[]` en même temps et corrompre les données (race condition).

Un point subtil à noter : dans `save_clients()`, le mutex `mtx_clients` (pour le fichier) et `mtx_mem` (pour le tableau) sont tous deux verrouillés — l'ordre doit être **toujours le même** pour éviter un deadlock (verrou mortel).

Thread de réception côté client (`recv_thread`)



Le problème que `recv_thread` résout

Sans thread de réception, le client serait bloqué dans une boucle séquentielle : il enverrait une commande, attendrait la réponse, puis pourrait en envoyer une autre. Ça marcherait pour des échanges simples, mais le serveur peut envoyer des messages à **tout moment**

(par exemple un **UPDATE** envoyé par le jumeau numérique, ou un **MSG** d'un autre client). Il faut donc deux activités en parallèle.

Ce que fait concrètement **recv_thread**

```
static void *recv_thread(void *arg)
{
    int sockfd = *(int *)arg; // reçoit le descripteur de socket par pointeur
    char buf[BUF_SIZE];

    while (1) {                // boucle infinie
        memset(buf, 0, sizeof(buf));
        ssize_t n = recv(sockfd, buf, sizeof(buf) - 1, 0);
        // ↑ BLOQUANT : le thread dort ici jusqu'à ce que le serveur envoie quelque chose

        if (n <= 0) {           // n==0 = déconnexion propre, n<0 = erreur réseau
            printf("\n[Serveur déconnecté]\n");
            break;              // sort de la boucle → le thread se termine
        }

        printf("\n← %s", buf); // affiche la réponse serveur avec le préfixe ←
        printf("> ");           // réaffiche le prompt de saisie
        fflush(stdout);         // force l'affichage immédiat
    }
    return NULL;
}
```

Et dans **run_tcp()**, voici comment il est lancé :

```
pthread_t tid;
pthread_create(&tid, NULL, recv_thread, &sockfd);
// ↑ crée le thread, lui passe l'adresse de sockfd

pthread_detach(tid);
// ↑ "oublie" le thread : pas besoin de pthread_join()
// il se terminera seul quand le serveur se déconnecte
```

Points importants à retenir

recv() est **bloquant par défaut** : le thread de réception ne consomme pas de CPU quand il n'y a rien à recevoir, il dort simplement dans le noyau jusqu'à ce que des données arrivent sur la socket.

La **socket est partagée** entre les deux threads (le principal qui appelle `send()`, et `recv_thread` qui appelle `recv()`). En TCP, envoyer et recevoir sont deux canaux indépendants (full-duplex), donc il n'y a pas de conflit d'accès ici — pas besoin de mutex pour la socket.

Le `pthread_detach` signifie que le thread de réception est autonome. Quand le serveur coupe la connexion (`n <= 0`), il affiche le message et sort proprement, sans que le thread principal ait besoin de l'attendre.

Le `fflush(stdout)` après le réaffichage du `>` est indispensable : sans lui, le texte resterait dans le buffer et n'apparaîtrait pas immédiatement dans le terminal, surtout quand la sortie n'est pas un TTY.